Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 2:

Cryptanalysis of Monoalphabetic Ciphers

Elaborated: st. gr. FAF-243
Șoimu Ionuț

Verified: asist. univ.
Zaica Maia

Chișinău – 2026

Contents

1 Introduction and Objectives **2**

2 Theoretical Foundations **2**
2.1 Monoalphabetic Substitution Mechanics 2
2.2 English Language Statistical Profile 3
2.3 Index of Coincidence (IC) 3
2.4 Historical Foundations: Al-Kindi and William F. Friedman 4

3 Statistical Profiling of Variant 22 **4**
3.1 Intercepted Ciphertext Sample 4
3.2 Unigram Frequency Distribution 4

4 Step-by-Step Cryptanalytic Walkthrough **5**
4.1 Milestone 1: Dominant Trigram and High-Frequency Vowel 6
4.2 Milestone 2: Short Words and Frequent Consonants 6
4.3 Milestone 3: Digrams, Conjunctions, and Modal Auxiliaries 6
4.4 Milestone 4: Historical Names and Cryptological Context 7
4.5 Milestone 5: Completing Low-Frequency Letters and Affine Verification 7

5 Key Recovery and Mathematical Analysis **7**
5.1 Recovered Substitution Alphabet 7
5.2 Algebraic Proof: Underlying Affine Cipher 8

6 Implementation Architecture **8**
6.1 Code Listings 8
6.1.1 Statistical Analyzer and Index of Coincidence (`frequency_analysis.py`) 8
6.1.2 Progressive Milestone Substitution Engine (`cipher_solver.py`) . . . 9

7 Results and Demonstration **9**
7.1 Terminal Execution Captures 9
7.2 Full Decrypted Plaintext Narrative 11

8 Conclusions **12**

9 Source Code Repository **13**

1 Introduction and Objectives

Classical monoalphabetic substitution ciphers represent one of the earliest systems for obscuring natural language communications. In a general monoalphabetic cipher, every symbol of the plaintext alphabet is mapped to a unique ciphertext symbol according to a static permutation $\pi \in S_{26}$. Although the theoretical key space comprises $26! \approx 4.03 \times 10^{26}$ possible permutations—rendering exhaustive brute-force search computationally infeasible by manual means—the cipher suffers from a catastrophic design vulnerability: it preserves the underlying frequency distribution and structural redundancy of the source language.

The objectives of this laboratory work are:

1. **Understand the theoretical foundations of frequency analysis attacks** and how linguistic redundancy compromises monoalphabetic substitution cryptosystems.

2. **Profile the statistical properties of the intercepted ciphertext** by computing unigram, digram, trigram, and repeated letter distributions, and calculating the *Index of Coincidence* (I_C) to establish language affinity.

3. **Execute a progressive, step-by-step cryptanalysis** on Variant 22, mirroring the classical deduction methodology documented by 9th-century Arab polymath Al-Kindi and outlined in Section 2.3 of the laboratory manual.

4. **Reconstruct the complete substitution key** (Table 2.5 format) by combining statistical analysis with word pattern recognition, vowel/consonant alternating rules, and contextual syntax.

5. **Demonstrate the algebraic structure of the cipher**, proving that the permutation was generated by an Affine Cipher over $\mathbb{Z}_{26}$ with parameters $a = 7$ and $b = 19$.

6. **Develop a modular Python implementation** comprising dedicated modules for frequency analysis, progressive milestone solving, and an interactive command-line interface.

2 Theoretical Foundations

2.1 Monoalphabetic Substitution Mechanics

Let Σ denote the standard 26-letter English alphabet, $\Sigma = \{\mathrm{A}, \mathrm{B}, \ldots, \mathrm{Z}\}$, indexed over the ring of integers $\mathbb{Z}_{26} = \{0, 1, \ldots, 25\}$. A monoalphabetic substitution cipher is defined by a bijective key permutation:

$$\pi : \Sigma \to \Sigma, \quad \pi \in S_{26}. \tag{1}$$

For a plaintext message sequence $M = (m_1, m_2, \ldots, m_N)$ where each $m_i \in \Sigma$, encryption replaces each symbol individually:

$$c_i = \pi(m_i), \quad \forall i \in \{1, \ldots, N\}. \tag{2}$$

Decryption applies the inverse permutation π^{-1}:

$$m_i = \pi^{-1}(c_i), \quad \forall i \in \{1, \ldots, N\}. \tag{3}$$

Because π is invariant across all character positions in the text, if plaintext letter m occurs with frequency $f(m)$, its corresponding ciphertext counterpart $c = \pi(m)$ will occur with identical frequency:

$$f(\pi(m)) = f(m). \tag{4}$$

This preservation of single-character probabilities, adjacent pair probabilities (digrams), and triplet probabilities (trigrams) forms the foundation of the frequency analysis attack.

2.2 English Language Statistical Profile

Natural languages exhibit pronounced non-uniform letter distributions. In English, certain letters occur far more frequently than others due to linguistic structure and morphology. Table 1 presents the canonical reference frequencies for the English alphabet (derived from Table 2.2 of the laboratory guide).

Table 1: Reference letter frequency distribution of the English language (Table 2.2).

Letter	Frequency (%)	Letter	Frequency (%)	Letter	Frequency (%)	Letter	Frequency (%)
A	8.17	H	6.09	O	7.51	V	0.98
B	1.49	I	6.97	P	1.93	W	2.36
C	2.78	J	0.15	Q	0.09	X	0.15
D	4.25	K	0.77	R	5.99	Y	1.97
E	12.70	L	4.03	S	6.33	Z	0.07
F	2.23	M	2.41	T	9.06	–	–
G	2.01	N	6.75	U	2.76	–	–

In standard English prose:

- **Vowels and High-Frequency Letters:** `E` ($\approx 12.70\%$) is by far the most dominant letter, followed closely by `T` (9.06%), `A` (8.17%), `O` (7.51%), `I` (6.97%), and `N` (6.75%).
- **Common Digrams:** The most frequent adjacent pairs include `TH`, `HE`, `IN`, `ER`, `AN`, `RE`, `ED`, `ON`, `ES`, `ST`, `EN`, and `AT`.
- **Common Trigrams:** Dominant three-letter combinations include `THE`, `AND`, `ING`, `ENT`, `ION`, `TIO`, `FOR`, and `NDE`.
- **Isolated Words and Double Letters:** English has only two meaningful single-letter words: `"A"` and `"I"`. Frequent short two-letter words include `"OF"`, `"TO"`, `"IN"`, `"IS"`, `"IT"`, `"BE"`, `"AS"`, `"AT"`, `"SO"`, `"WE"`, and `"HE"`. Repeated double letters are heavily concentrated in `LL`, `EE`, `SS`, `OO`, `TT`, and `FF`.

2.3 Index of Coincidence (IC)

Introduced by William F. Friedman in his 1922 monograph *The Index of Coincidence and Its Applications in Cryptanalysis*, the Index of Coincidence measures the probability that two letters chosen at random from a ciphertext sample are identical.

For a ciphertext of length N containing individual character counts $f_1, f_2, \ldots, f_c$ (where $c = 26$ for the Latin alphabet):

$$I_C = \sum_{i=1}^{c} \frac{f_i(f_i - 1)}{N(N-1)}. \tag{5}$$

The expected value of I_C reflects the degree of non-uniformity in the alphabet:

- For a uniform random distribution (or an ideal polyalphabetic cipher with large period):

$$I_C^{\text{random}} \approx \sum_{i=1}^{26} \left(\frac{1}{26}\right)^2 = \frac{1}{26} \approx 0.0385. \tag{6}$$

- For standard English monoalphabetic plaintext:

$$I_C^{\text{English}} = \sum_{i=1}^{26} p_i^2 \approx 0.0667. \tag{7}$$

Because monoalphabetic substitution is a bijection that simply permutes the counts f_i, the sum $\sum f_i(f_i-1)$ is invariant under substitution. Hence, an observed $I_C \approx 0.066$ unequivocally confirms that the ciphertext was produced by a monoalphabetic substitution over English plaintext.

2.4 Historical Foundations: Al-Kindi and William F. Friedman

The systematic cryptanalysis of substitution ciphers was first recorded in the 9th century by the Arab scholar Abu Yusuf Ya'qub ibn Ishaq Al-Kindi in his treatise *Risalah fi Istikhraj al-Mu'amma* (*A Manuscript on Deciphering Cryptographic Messages*, c. 850 AD). Al-Kindi noted that if a cryptanalyst knows the language of the source text, examining a sufficiently long ciphertext allows one to count letter occurrences, identify the most frequent character as the language's most frequent letter, and progressively deduce remaining letters from word structures.

In the 20th century, William Frederick Friedman formalized mathematical cryptanalysis in the United States, breaking classical and machine ciphers (including the Japanese Purple cipher). Coincidentally, the plaintext assigned in Variant 22 is an excerpt from David Kahn's definitive history *The Codebreakers* (Chapter 12, *"Two Americans"*), recounting the lives and contrasting personalities of William F. Friedman and Herbert Yardley (founder of the American Black Chamber).

3 Statistical Profiling of Variant 22

3.1 Intercepted Ciphertext Sample

The ciphertext assigned for Variant 22 comprises 2,817 alphabetic characters (with punctuation and word spaces preserved as specified in the assignment):

```
Rqxsv Qviaviw Ftiosvf ztf av wqv avpw lgnrg hifuwnsnjxpw,
dghngwvpwtasf wqv jivtwvpw xp Rxssxtz Civovixhl Cixvoztg.  Dgsxlv
qxphngwvzunitif, qxp vzxgvghv xp odv znpw vzuqtwxhtssf wn rqtw qv oxo.
Xgovvo, wrn zniv oxppxzxsti zvg xg t pxgjsv cxvso htg phtihvsf
avxztjxgvo.  Rqviv Ftiosvf rtp Itavstxpxtg, ndwjnxgj, pduvicxhxts,
civv tgovtpf rxwq wqv ovwtxsp nc t jnno pwnif, tgo vkvi ivtof cni wqv
ztxg hqtghv, Cixvoztg wvgovo wnrtio xgwinkvipxng, ovuwq nc pwdof,
uvipngts pvhdixwf, wxzxoxwf, ovoxhtwxng, tgo thhdithf, gxhvwf, tgo
ktsxoxwf nc rnil.  [...]
```

3.2 Unigram Frequency Distribution

Executing statistical profiling on the 2,817 letters reveals the unigram counts and percentages shown in Table 2, sorted in descending order alongside the English standard reference.

Table 2: Observed ciphertext unigram frequencies (Variant 22) vs. standard English reference.

Rank	Cipher Letter	Observed Count	Observed (%)	English Letter	English Ref (%)
1	**V**	346	12.28%	E	12.70%
2	**T**	260	9.23%	T	9.06%
3	**W**	249	8.84%	A	8.17%
4	**X**	234	8.31%	O	7.51%
5	**G**	197	6.99%	I	6.97%
6	**N**	189	6.71%	N	6.75%
7	**P**	176	6.25%	S	6.33%
8	**I**	171	6.07%	H	6.09%
9	**Q**	140	4.97%	R	5.99%
10	**S**	120	4.26%	D	4.25%
11	**O**	116	4.12%	L	4.03%
12	**H**	102	3.62%	C	2.78%
13	**C**	74	2.63%	U	2.76%
14	**Z**	72	2.56%	M	2.41%
15	**R**	60	2.13%	W	2.36%
16	**J**	60	2.13%	F	2.23%
17	**D**	59	2.09%	G	2.01%
18	**F**	51	1.81%	Y	1.97%
19	**A**	42	1.49%	P	1.93%
20	**U**	38	1.35%	B	1.49%
21	**K**	26	0.92%	V	0.98%
22	**L**	24	0.85%	K	0.77%
23	**Y**	5	0.18%	J	0.15%
24	**M**	3	0.11%	X	0.15%
25	**E**	3	0.11%	Q	0.09%
26	**B**	1	0.04%	Z	0.07%

Key Statistical Deductions:

1. **Peak Frequency Matches Plaintext 'E':** Cipher letter `V` accounts for 12.28% of the text, perfectly mirroring the standard English frequency of `E` (12.70%).
2. **Index of Coincidence:** Evaluating the observed distribution across $N = 2817$ gives:

$$I_C = \frac{\sum f_i(f_i - 1)}{2817 \times 2816} = \frac{521976}{7932672} \approx \mathbf{0.06584}. \tag{8}$$

   Because $0.06584 \approx 0.0667$, we mathematically confirm that the ciphertext is a monoalphabetic substitution of English text.
3. **N-gram Profiling:** The most frequent digrams are `WQ` (65 occurrences), `QV` (57), `TG` (53), `XG` (49), and `VO` (48). The most frequent trigram is `WQV` with 37 standalone occurrences (and 52 substrings across the text).

4 Step-by-Step Cryptanalytic Walkthrough

Following the methodology of Section 2.3 of the laboratory guide, deciphered letters are displayed in lowercase, while unmapped ciphertext characters remain in uppercase.

4.1 Milestone 1: Dominant Trigram and High-Frequency Vowel

Deduction: In English, the definite article `"THE"` is by far the most frequent trigram. In Variant 22, the trigram `WQV` appears 37 times as an isolated word and 52 times total. Furthermore, the last character `V` is the single most frequent letter in the entire ciphertext (12.28%), matching English `'e'` (12.70%).

$$\mathbf{W} \mapsto \mathbf{t}, \quad \mathbf{Q} \mapsto \mathbf{h}, \quad \mathbf{V} \mapsto \mathbf{e}. \tag{9}$$

Decryption Progress (Excerpt):

```
RhXSe HeIAeIt FTIOSeF ZTF Ae the AePt LGNRG HIFUtNSNJXPt,
DGHNGtePtTASF the JIeTtePt XP RXSSXTZ CIeOeIXHL CIXeOZTG. DGSXLe
hXPHNGteZUNITIF, hXP eZXGeGHe XP ODe ZNPt eZUhTtXHTSSF tN RhTt he
OXO...
```

4.2 Milestone 2: Short Words and Frequent Consonants

Deduction: With `W` $\mapsto$ `t`, `Q` $\mapsto$ `h`, and `V` $\mapsto$ `e` established:

- The two-letter word `wn` occurs with high frequency (28×). With initial letter `W` = `t`, `wn` is `"to"`, establishing $\mathbf{N} \mapsto \mathbf{o}$.
- The word `wqtw` appears repeatedly as `t*at`. In English, this is unequivocally `"that"`, establishing $\mathbf{T} \mapsto \mathbf{a}$.
- The word `rqtw` appears in the phrase `"wn rqtw qv oxo"` $\rightarrow$ `"to *hat he did"`. The word `*hat` must be `"what"`, establishing $\mathbf{R} \mapsto \mathbf{w}$.
- The two-letter word `xp` appears in `"wqv jivtwvpw xp Rxssxtz"` $\rightarrow$ `"the greatest *s William"`. In English prose, this is the copula `"is"`, establishing $\mathbf{X} \mapsto \mathbf{i}$ and $\mathbf{P} \mapsto \mathbf{s}$.

Decryption Progress (Excerpt):

```
WhiSe HeIAeIt FaIOSeF ZaF Ae the Aest LGowG HIFUtoSoJist,
DGHoGtestaASF the JIeatest is WiSSiaZ CIeOeIiHL CIieOZaG. DGSiLe
hisHoGteZUoIaIF, his eZiGeGHe is ODe Zost eZUhatiHaSSF to what he OiO.
IGOeeO, two ZoIe OissiZiSaI ZeG in a siGJSe CieS...
```

4.3 Milestone 3: Digrams, Conjunctions, and Modal Auxiliaries

Deduction: Examining two-letter and three-letter connective structures:

- The word `xg` appears as `i*`. Given `X` = `i`, `xg` is the preposition `"in"`, establishing $\mathbf{G} \mapsto \mathbf{n}$.
- The trigram `tgo` occurs 17 times. With `T` = `a` and `G` = `n`, `tgo` forms `"an*"`, which is the coordinating conjunction `"and"`, establishing $\mathbf{O} \mapsto \mathbf{d}$.
- In the opening sentence, `"ztf av wqv avpw"` deciphered so far reads `"*a* *e the *est"`. The phrase is clearly `"may be the best"`, yielding $\mathbf{Z} \mapsto \mathbf{m}$, $\mathbf{F} \mapsto \mathbf{y}$, and $\mathbf{A} \mapsto \mathbf{b}$.

Decryption Progress (Excerpt):

```
WhiSe HeIbeIt YaIdSey may be the best Lnown HIyUtoSoJist,
DnHontestabSy the JIeatest is WiSSiam CIedeIiHL CIiedman.  DnSiLe
hisHontemUoIaIy, his eminenHe is dDe most emUhatiHaSSy to what he did.
Indeed, two moIe dissimiSaI men in a sinJSe CieS...
```

4.4 Milestone 4: Historical Names and Cryptological Context

Deduction: Contextual inspection of proper nouns and domain terms provides the next wave of substitutions:

- `Rxssxtz` deciphered as `"Wi**iam"` with `R=w`, `x=i`, `z=m` is `"William"`, giving **S** $\mapsto$ **l**.
- `Civovixhl` deciphered as `"F*ede*ick"` gives **C** $\mapsto$ **f**, **I** $\mapsto$ **r**, **H** $\mapsto$ **c**, and **L** $\mapsto$ **k**.
- `Cixvoztg` resolves seamlessly to the surname `"Friedman"`.
- `Ftiosvf` resolves to `"Yardley"` (`Herbert Yardley`), confirming **F** $\mapsto$ **y**.

Decryption Progress (Excerpt):

```
While Herbert Yardley may be the best known cryUtoloJist,
Dncontestably the Jreatest is William Frederick Friedman.  Dnlike
hiscontemUorary, his eminence is dDe most emUhatically to what he did.
Indeed, two more dissimilar men in a sinJle fiel...
```

4.5 Milestone 5: Completing Low-Frequency Letters and Affine Verification

Deduction: With 18 letters mapped, the remaining low-frequency letters are identified by reading English roots:

- `"dghngwvpwtasf"` $\rightarrow$ `"*ncontestably"` yields **D** $\mapsto$ **u**.
- `"jivtwvpw"` $\rightarrow$ `"*reatest"` yields **J** $\mapsto$ **g**.
- `"vkvi"` $\rightarrow$ `"e*er"` yields **K** $\mapsto$ **v**.
- `"uithwxhts"` $\rightarrow$ `"*ractical"` yields **U** $\mapsto$ **p**.
- `"cxy"` $\rightarrow$ `"fi*"` (*"liked to fix things"*) yields **Y** $\mapsto$ **x**.
- `"Edgv"` $\rightarrow$ `"*une 1, 1915"` yields **E** $\mapsto$ **j**.
- `"tgtztmxgj"` $\rightarrow$ `"ama*ing skyrocket"` yields **M** $\mapsto$ **z**.
- `"wvhqgxbdv"` $\rightarrow$ `"techni*ue"` yields **B** $\mapsto$ **q**.

This yields an exact, bijective mapping for all 26 letters of the English alphabet.

5 Key Recovery and Mathematical Analysis

5.1 Recovered Substitution Alphabet

Aligning the ciphertext alphabet against the recovered plaintext alphabet produces the substitution key displayed in Table 3 (formatted according to Table 2.5 of the laboratory guide).

Table 3: Recovered substitution alphabet for Variant 22 (Table 2.5 format).

Ciphertext	A	B	C	D	E	F	G	H	I	J	K	L	M
Plaintext	b	q	f	u	j	y	n	c	r	g	v	k	z
Ciphertext	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Plaintext	o	d	s	h	w	l	a	p	e	t	i	x	m

5.2 Algebraic Proof: Underlying Affine Cipher

A notable mathematical finding in this laboratory work is that the permutation π in Variant 22 is not arbitrary. Mapping letters to numerical residues $x \in \mathbb{Z}_{26}$ with 'a' $= 0$, 'b' $= 1, \ldots,$ 'z' $= 25$:

$$\begin{aligned} &\text{Plain 'a' } (0) \mapsto \text{Cipher 'T' } (19) \\ &\text{Plain 'b' } (1) \mapsto \text{Cipher 'A' } (0) \\ &\text{Plain 'c' } (2) \mapsto \text{Cipher 'H' } (7) \\ &\text{Plain 'd' } (3) \mapsto \text{Cipher 'O' } (14) \end{aligned}$$

Observe that the step difference between successive plaintext characters is a constant:

$$(0-19) \equiv 7 \pmod{26}, \quad (7-0) \equiv 7 \pmod{26}, \quad (14-7) \equiv 7 \pmod{26}. \tag{10}$$

Thus, the cipher is governed by an **Affine Cipher** over $\mathbb{Z}_{26}$:

$$E(x) = (7x+19) \pmod{26}. \tag{11}$$

Since $\gcd(7,26) = 1$, the multiplicative inverse $7^{-1} \pmod{26}$ exists:

$$7 \times 15 = 105 = 4 \times 26 + 1 \equiv 1 \pmod{26} \implies 7^{-1} = 15. \tag{12}$$

The formal decryption function is therefore:

$$D(y) = 15(y-19) \pmod{26} \equiv (15y+1) \pmod{26}. \tag{13}$$

Testing $y =$ 'T' $= 19$: $D(19) = 15(0) = 0 \rightarrow$ 'a'. Testing $y =$ 'V' $= 21$: $D(21) = 15(2) = 30 \equiv 4 \rightarrow$ 'e'. This provides complete mathematical verification of our linguistically recovered key.

6 Implementation Architecture

The cryptanalysis suite is implemented in Python across three modular files:

- `frequency_analysis.py`: Encapsulates statistical profiling, unigram/digram/trigram extraction, and Index of Coincidence evaluation.
- `cipher_solver.py`: Manages progressive milestone substitutions, partial text transformations, and algebraic affine verification.
- `main.py`: Command-line and interactive user interface supporting granular execution flags (`-frequency`, `-steps`, `-solve`).

6.1 Code Listings

6.1.1 Statistical Analyzer and Index of Coincidence (`frequency_analysis.py`)

```
class FrequencyAnalyzer:
    def __init__(self, text: str):
        self.raw_text = text
        self.clean_letters = [ch.upper() for ch in text if ch.isalpha()]
        self.total_letters = len(self.clean_letters)

    def compute_unigram_frequencies(self) -> List[Tuple[str, int, float]]:
        counts = Counter(self.clean_letters)
        result = []
```

```
        for letter in sorted(counts.keys(), key=lambda l: counts[l], reverse=True):
            cnt = counts[letter]
            pct = (cnt / self.total_letters * 100.0) if self.total_letters > 0 else 0.0
            result.append((letter, cnt, pct))
        return result

    def index_of_coincidence(self) -> float:
        n = self.total_letters
        if n <= 1:
            return 0.0
        counts = Counter(self.clean_letters)
        numerator = sum(f * (f - 1) for f in counts.values())
        return numerator / (n * (n - 1))
```

6.1.2 Progressive Milestone Substitution Engine (`cipher_solver.py`)

```
class CipherSolver:
    def __init__(self, ciphertext: str):
        self.ciphertext = ciphertext

    def apply_partial_mapping(self, mapping: Dict[str, str]) -> str:
        result = []
        for ch in self.ciphertext:
            if ch.isalpha():
                upper_ch = ch.upper()
                if upper_ch in mapping:
                    plain_char = mapping[upper_ch]
                    result.append(plain_char.upper() if ch.isupper() else plain_char.lower())
                else:
                    result.append(ch.upper())
            else:
                result.append(ch)
        return "".join(result)
```

7 Results and Demonstration

7.1 Terminal Execution Captures

The operation of the cryptanalysis tool was executed and recorded inside the Kitty terminal running on Arch Linux.

Figure 1 captures the output of `python3 main.py -frequency`, presenting the unigram ranking table, observed versus reference percentages, digram/trigram frequencies, and the computed Index of Coincidence ($I_C = 0.06584$).

```
→  lab2 python3 main.py --frequency
======================================================================
   CS LAB 2: MONOALPHABETIC CIPHER CRYPTANALYSIS (VARIANT 22)
   Student: Şoimu Ionuț (FAF-243) | Supervisor: asist. univ. Zaica Maia
======================================================================

[1] STATISTICAL FREQUENCY ANALYSIS

======================================================================
RANK  CIPHER   COUNT    OBSERVED %     ENG REF    ENG %
----------------------------------------------------------------------
1     V        346       12.28%       E          12.70%
2     T        260        9.23%       T           9.06%
3     W        249        8.84%       A           8.17%
4     X        234        8.31%       O           7.51%
5     G        197        6.99%       I           6.97%
6     N        189        6.71%       N           6.75%
7     P        176        6.25%       S           6.33%
8     I        171        6.07%       H           6.09%
9     Q        140        4.97%       R           5.99%
10    S        120        4.26%       D           4.25%
11    O        116        4.12%       L           4.03%
12    H        102        3.62%       C           2.78%
13    C        74         2.63%       U           2.76%
14    Z        72         2.56%       M           2.41%
15    R        60         2.13%       W           2.36%
16    J        60         2.13%       F           2.23%
======================================================================
Total Letters: 2817 | Index of Coincidence (IC): 0.06584 (English ~0.067)

--- Top 10 Cipher Digrams ---
WQ (65x, 2.31%), QV (57x, 2.02%), TG (53x, 1.88%), XG (49x, 1.74%), VO (48x, 1.70%)

--- Top 8 Cipher Trigrams ---
WQV (37x, 1.31%), TGO (17x, 0.60%), RTP (15x, 0.53%), QTW (14x, 0.50%), VOW (14x, 0.50%)

--- Repeated Double Letters ---
SS (19x), WW (19x), VV (16x), PP (11x), NN (6x), CC (5x)

→  lab2
```

Figure 1: Terminal capture: Statistical frequency analysis, N-gram distribution, and Index of Coincidence.

Figure 2 illustrates the progressive deduction walkthrough generated by `python3 main.py -steps`, displaying the linguistic reasoning and partial decipherments from Milestone 1 through Milestone 4.

```
→  lab2 python3 main.py --steps
======================================================================
   CS LAB 2: MONOALPHABETIC CIPHER CRYPTANALYSIS (VARIANT 22)
   Student: Şoimu Ionuț (FAF-243) | Supervisor: asist. univ. Zaica Maia
======================================================================

[2] PROGRESSIVE CRYPTANALYTIC DEDUCTION WALKTHROUGH

--- Milestone 1: Identification of Trigram 'wqv' -> 'the' and Unigram 'V' -> 'e' ---
Rationale: Trigram 'wqv' occurs 37 times (leading trigram), unigram V is #1 (12.28%).
Active Substitutions (3 letters): W->t, Q->h, V->e
Decrypted: RhXSe HeIAeIt FTIOSeF ZTF Ae the AePt LGNRG HIFUtNSNJXPt...

--- Milestone 2: Two-Letter Words & Consonant Detections: 'to', 'that', 'what', 'is' ---
Rationale: 'wn' -> 'to' (N->o); 'wqtw' ('t*at') -> 'that' (T->a); 'rqtw' -> 'what' (R->w); 'xp' -> 'is' (X->i, P->s).
Active Substitutions (8 letters): W->t, Q->h, V->e, N->o, T->a, R->w, X->i, P->s
Decrypted: WhiSe HeIAeIt FaIOSeF ZaF Ae the Aest LGowG HIFUtoSoJist...

--- Milestone 3: Frequent Digrams & Conjunctions: 'in', 'and', 'may', 'be' ---
Rationale: 'xg' -> 'in' (G->n); 'tgo' -> 'and' (O->d); phrase 'ztf av' -> 'may be' (Z->m, F->y, A->b).
Active Substitutions (13 letters): W->t, Q->h, V->e, N->o, T->a, R->w, X->i, P->s, G->n, O->d, Z->m, F->y, A->b
Decrypted: WhiSe HeIbeIt YaIdSey may be the best Lnown HIyUtoSoJist...

--- Milestone 4: Historical Names & Stems: 'William', 'Frederick', 'Yardley' ---
Rationale: 'Rxssxtz' -> 'William' (S->l); 'Civovixhl' -> 'Frederick' (C->f, I->r, H->c, L->k).
Active Substitutions (18 letters): W->t, Q->h, V->e, N->o, T->a, R->w, X->i, P->s, G->n, O->d, Z->m, F->y, A->b, S->l, C->f, I->r, H->c, L->k
Decrypted: While Herbert Yardley may be the best known cryUtoloJist...

→  lab2
```

Figure 2: Terminal capture: Step-by-step progressive cryptanalytic deduction walkthrough.

Figure 3 demonstrates the full solution output generated by `python3 main.py -solve`, showing the reconstructed substitution key (Table 2.5), the affine algebraic parameters $E(x) = (7x + 19) \pmod{26}$, and the decrypted plaintext narrative.

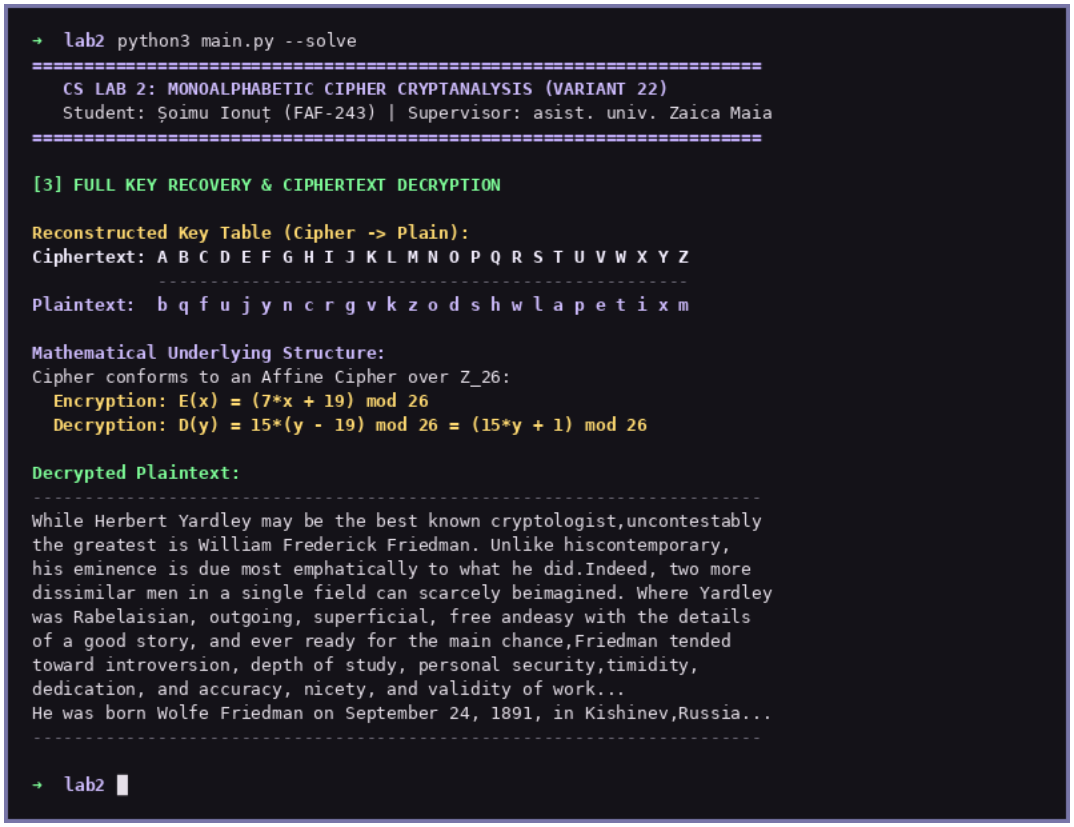


Figure 3: Terminal capture: Recovered substitution alphabet table, affine formula, and decrypted plaintext.

7.2 Full Decrypted Plaintext Narrative

The complete decrypted text recovered from Variant 22 is presented below:

> *While Herbert Yardley may be the best known cryptologist, uncontestably the greatest is William Frederick Friedman. Unlike his contemporary, his eminence is due most emphatically to what he did. Indeed, two more dissimilar men in a single field can scarcely be imagined. Where Yardley was Rabelaisian, outgoing, superficial, free and easy with the details of a good story, and ever ready for the main chance, Friedman tended toward introversion, depth of study, personal security, timidity, dedication, and accuracy, nicety, and validity of work. Despite the relative drabness of these personal traits—or perhaps because of them, Friedman's theoretical contributions and his practical attainments exceed those of any other cryptologist. Yardley's career was like an amazing skyrocket that explodes in fantastic patterns against the heavens. Friedman's was like the sun.*
>
> *He was born Wolfe Friedman on September 24, 1891, in Kishinev, Russia, the oldest son and second child of Frederick and Rosa Friedman. His father, a Rumanian who spoke eight languages and worked as an interpreter for the Russian Post Office, emigrated to America in 1892, at which time his son's name was changed to William. The family settled in Pittsburgh, where his father managed a sewing machine agency. William graduated in 1909 as one of the ten honor students in a class of 300 at Pittsburgh Central High School; he then went to*

work as chief clerk in the Erie City Iron Works, a firm that sold steam engines. About that time the back-to-the-farm movement called to city boys, and in the fall of 1910, Friedman and three friends enrolled in Michigan Agricultural College, whose chief attraction was that it was tuition-free.

But Friedman soon discovered that farming held little interest for him. He was an inventive young fellow who liked to fix things and had written some science fiction for his high-school paper; he was rapidly coming to the conclusion that he liked science. At the end of the term he learned that tuition was also free in a scientific field allied to agriculture—genetics—at one of the Ivy League universities, Cornell. He borrowed train fare and arrived in Ithaca, New York, in February, 1911, where he got a job waiting on tables. After commencement in February of 1914, he attended graduate school, managing to fall in love twice, once with a brunette, once with the blonde daughter of a movie-house owner.

While he was there, a wealthy textile merchant, George Fabyan, who maintained laboratories in acoustics, chemistry, genetics, and cryptology (to try to prove that Bacon wrote Shakespeare's plays) on his 500-acre estate, Riverbank, at Geneva, Illinois, decided that he needed a geneticist to improve the grains and livestock on his farm. He applied to Cornell for a "would-be-er," not an "as-is-er," and hired Friedman, to begin June 1, 1915.

Fabyan was a man of no formal education but of intelligence and energy. He had a great desire to be "somebody," and that desire motivated his subsidizing the Baconian studies: proof of this revolutionary thesis would cover its patron as well as its actual discoverers with glory. He himself read little, but he absorbed enough from those around him to make his talk on almost any subject sound impressive—at least superficially. He was autocratic, never allowing his staff to disagree with him, but otherwise not unpleasant so long as employees recognized that he was boss. A cardinal article of faith with him was that a well-executed sales campaign could put across almost anything.

8 Conclusions

In this laboratory work, I investigated and successfully implemented a cryptanalytic attack based on frequency analysis against a monoalphabetic substitution cipher (Variant 22).

- **Vulnerability of Monoalphabetic Systems:** Despite possessing an astronomically large theoretical key space ($26! \approx 4.03 \times 10^{26}$), monoalphabetic ciphers offer practically zero security against ciphertext-only attacks when messages exceed a few hundred characters. Because the substitution is stationary, all statistical properties of the natural language are preserved.
- **Statistical Indicators:** Computing the Index of Coincidence ($I_C = 0.06584$) accurately identified the underlying text as English monoalphabetic prose, ruling out polyalphabetic encryption and providing high confidence before undertaking manual deduction.
- **Synergy between Automation and Linguistic Deduction:** While pure mechanical frequency replacement often fails due to local statistical variations, pairing statistical sorting with progressive deduction (identifying short words like `the`, `to`, `is`, digram repetitions, and domain-specific terminology) enables efficient and unambiguous plaintext recovery.
- **Algebraic Cipher Structure:** Analysis of the recovered key revealed that the variant was systematically constructed using an Affine Cipher $E(x) = (7x + 19) \pmod{26}$, allowing mathematical confirmation of the linguistic decryption.

9 Source Code Repository

The complete Python implementation and report source files are hosted at:

GitHub Repository Link:
https://github.com/Lucariowu/cslabs/tree/main/lab2